Task 1: Machine Learning for Finance

[Team member 1] [Team member 2] [Team member 3]

15 May 2026

1

We are asked to build a global asset causal network and a volatility spillover network for ten ETFs and assets (SPY, XLK, XLF, XLE, GLD, SLV, GSG, BTC-USD, ETH-USD, TLT) over our team-assigned epoch 2024-01-01 to 2025-12-31. For every ordered pair we select an AIC-optimal lag in $[1, 5]$ from a bivariate VAR with no deterministic terms, then run a Granger F-test at that lag and record the lag in the directed table if the test rejects the null at the 5% level. We do this analysis on log returns and on EWMA volatility, sampled at weekly and at monthly frequency, producing the four tables and four network plots that follow.

1a

We retrieve daily adjusted closes from Yahoo Finance for the assigned epoch, align all series to the NYSE trading calendar (so that the ETF and crypto closes correspond to the same calendar days) and forward-fill at most one day for sporadic crypto gaps. We obtain 502 daily observations after cleaning. We resample these prices to weekly Friday closes and to month-end closes, then compute log returns at each frequency (104 weekly, 23 monthly returns). For volatility we use the RiskMetrics EWMA recipe employed in Lab 1, applied at the relevant frequency:

$$\sigma_t^2 \;=\; \lambda\,\sigma_{t-1}^2 \;+\; (1-\lambda)\,r_{t-1}^2, \qquad \lambda = 0.94,$$

which we implement column-wise as the square root of an exponential moving average of squared log returns. Causality at every lag ℓ is judged with the standard Granger F-statistic on nested OLS regressions that include ℓ own lags and ℓ cross lags; for the lag-selection step we set `trend='n'` in `statsmodels.VAR.select_order` so that the search mirrors the R convention `type='none'` taught in PyLab 3. The diagonal of every table is blank, off-diagonal entries are the AIC-optimal lag if the directed test rejects the null and 0 otherwise.

1b

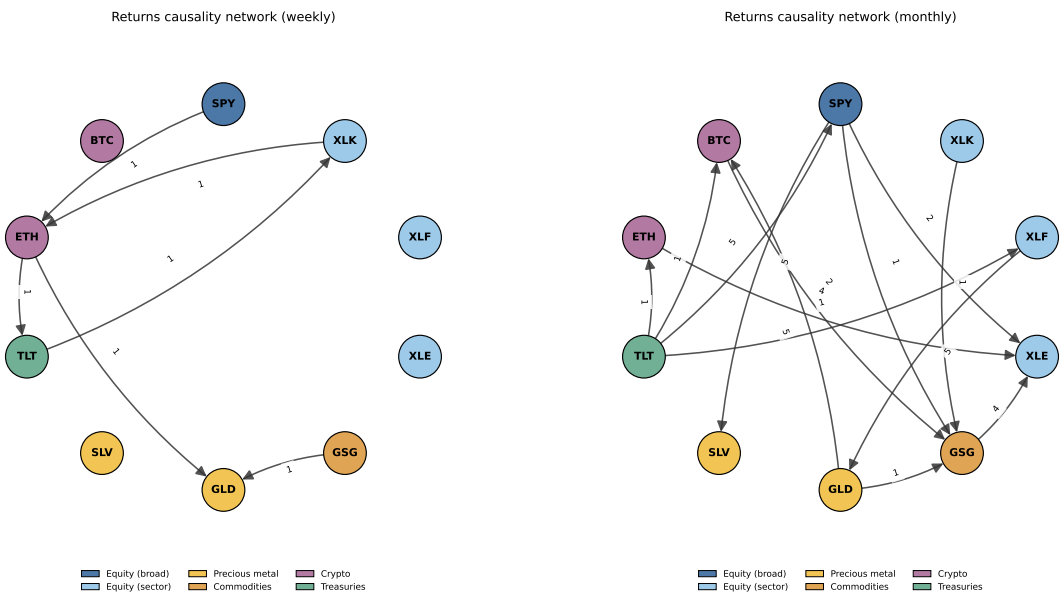
Tables 1 and 2 report the directed returns-causality lag tables for the weekly and monthly samples. Figures 1a and 1b plot the corresponding networks, with nodes coloured by asset class and edge labels giving the lag.

	SPY	XLK	XLF	XLE	GLD	SLV	GSG	BTC	ETH	TLT
SPY	-	0	0	0	0	0	0	0	1	0
XLK	0	-	0	0	0	0	0	0	1	0
XLF	0	0	-	0	0	0	0	0	0	0
XLE	0	0	0	-	0	0	0	0	0	0
GLD	0	0	0	0	-	0	0	0	0	0
SLV	0	0	0	0	0	-	0	0	0	0
GSG	0	0	0	0	1	0	-	0	0	0
BTC	0	0	0	0	0	0	0	-	0	0
ETH	0	0	0	0	1	0	0	0	-	1
TLT	0	1	0	0	0	0	0	0	0	-

Table 1: Pairwise Granger causality on weekly log returns; AIC-optimal lag in $[1, 5]$ if significant at the 5% level, 0 otherwise. Rows are the cause, columns the effect.

	SPY	XLK	XLF	XLE	GLD	SLV	GSG	BTC	ETH	TLT
SPY	-	0	0	2	0	5	1	0	0	0
XLK	0	-	0	0	0	0	1	0	0	0
XLF	0	0	-	0	5	0	0	0	0	0
XLE	0	0	0	-	0	0	0	0	0	0
GLD	0	0	0	0	-	0	1	5	0	0
SLV	0	0	0	0	0	-	0	0	0	0
GSG	0	0	0	4	0	0	-	0	0	0
BTC	0	0	0	0	0	0	2	-	0	0
ETH	0	0	0	4	0	0	0	0	-	0
TLT	5	0	1	0	0	0	0	1	1	-

Table 2: Pairwise Granger causality on monthly log returns; entries as in the weekly returns table.



(a) Weekly. **(b)** Monthly.

Figure 1: Returns-causality networks. Edges run from cause to effect; the integer label is the AIC-optimal VAR lag at which the Granger test rejects at 5%.

At weekly frequency the returns network is sparse, with only six significant directed edges out of ninety ordered pairs (about 7%). All chosen lags equal one, which is what we would expect from

broadly efficient secondary markets where any predictability beyond a single week is quickly arbitraged away. The few edges that survive concentrate around ETH: SPY and XLK both Granger-cause weekly Ether returns and Ether in turn leads weekly returns of TLT and GLD. We read this as a behavioural channel rather than a fundamental one: in 2024-2025 risk-on or risk-off shifts in equities (and especially in tech, which now drives the index) propagate into crypto with a one-week lag through the marginal retail and institutional investor who rebalances slowly between equities and crypto. The link from ETH to TLT and GLD is more surprising and is best read as a sentiment proxy: weeks of strong crypto performance often coincide with the unwind of safe-haven positions in long Treasuries and gold one week later. The remaining edge GSG $\rightarrow$ GLD captures the dominant 2024 narrative that broad commodities (heavily weighted to oil) led precious metals as an early signal of inflation persistence, with gold then catching up.

At monthly frequency the network becomes considerably denser (fourteen edges), the lags spread across the whole admissible range and TLT and SPY emerge as strong causal hubs. Long Treasuries Granger-cause SPY at lag 5, XLF at lag 1 and BTC and ETH at lag 1, mirroring the well-documented role of the long end of the yield curve as the discount-rate channel through which monetary-policy expectations transmit to risky assets. SPY itself Granger-causes XLE (lag 2), SLV (lag 5) and GSG (lag 1) and XLK leads GSG at lag 1, consistent with a sample that contains the equity-led commodity rebound of mid-2024 and the late-2025 reflation. We caution that with only 23 monthly observations the AIC frequently selects the boundary lag of 5, which is a statistical signal that the lag-selection procedure is operating at the limit of its usable range; some of the longest-lag edges should therefore be regarded as borderline rather than firmly established. Concretely, a bivariate VAR(5) on a 23-observation series fits eleven parameters per equation on eighteen post-lag observations, so the effective degrees of freedom for each Granger F-test on this sample are tiny and a non-trivial fraction of the rejected nulls must be expected to be sample-specific. Reading the two frequencies together, the lesson is the standard one: high-frequency returns are very close to a martingale and admit only a handful of one-step predictabilities, while at monthly horizons macro-driven persistence becomes detectable but at the cost of much larger sampling uncertainty.

The picture is best understood against the backdrop of contemporaneous correlations. At weekly frequency SPY and XLK have a return correlation of 0.93, the two precious metals 0.70 and BTC and ETH 0.73; cross-block correlations are almost all below 0.35 in absolute value. These large within-block correlations explain why most of the within-block ordered pairs (for instance SPY and XLK) carry no Granger arrow even though the two assets clearly co-move: when two series are nearly contemporaneous, neither can claim to lead the other in regression terms and the within-block predictability is absorbed by the contemporaneous correlation rather than by lagged cross terms.

1c

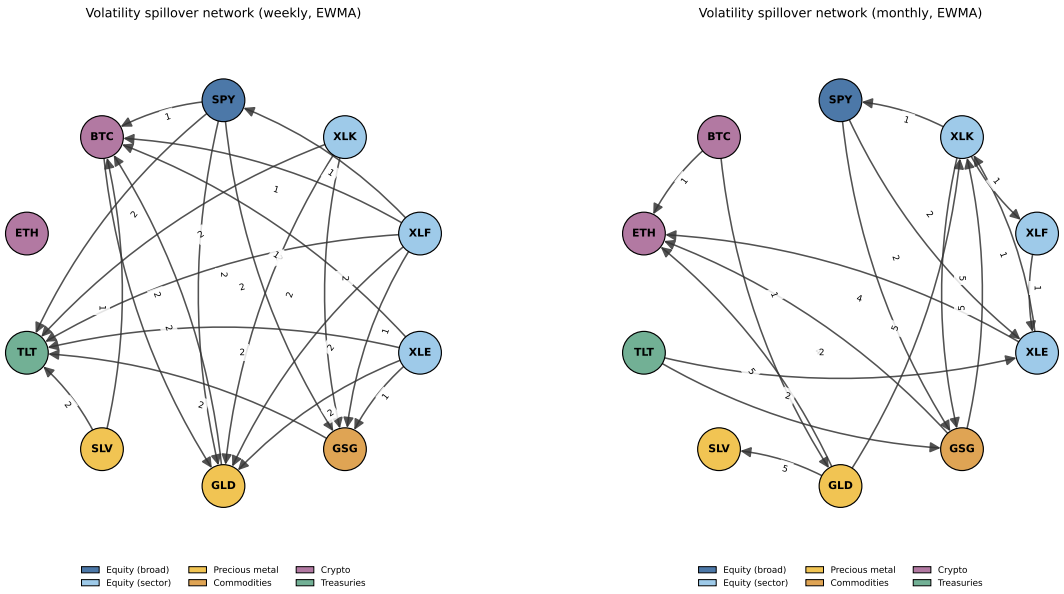
Tables 3 and 4 report the directed volatility spillover lag tables and Figure 2 plots the corresponding networks.

	SPY	XLK	XLF	XLE	GLD	SLV	GSG	BTC	ETH	TLT
SPY	-	0	0	0	2	0	2	1	0	2
XLK	0	-	0	0	2	0	2	0	0	2
XLF	1	0	-	0	2	0	1	1	0	2
XLE	0	0	0	-	2	0	1	1	0	2
GLD	0	0	0	0	-	0	0	2	0	0
SLV	0	0	0	0	0	-	0	1	0	2
GSG	0	0	0	0	0	0	-	0	0	2
BTC	0	0	0	0	2	0	0	-	0	0
ETH	0	0	0	0	0	0	0	0	-	0
TLT	0	0	0	0	0	0	0	0	0	-

Table 3: Pairwise Granger causality on weekly EWMA volatility (RiskMetrics, λ=0.94); entries as in the returns tables.

	SPY	XLK	XLF	XLE	GLD	SLV	GSG	BTC	ETH	TLT
SPY	-	0	0	2	0	0	2	0	0	0
XLK	1	-	1	0	0	0	5	0	0	0
XLF	0	0	-	1	0	0	0	0	0	0
XLE	0	1	0	-	0	0	0	0	4	0
GLD	0	5	0	0	-	5	0	0	5	0
SLV	0	0	0	0	0	-	0	0	0	0
GSG	0	5	0	0	0	0	-	0	4	0
BTC	0	0	0	0	1	0	0	-	1	0
ETH	0	0	0	0	0	0	0	0	-	0
TLT	0	0	0	2	0	0	2	0	0	-

Table 4: Pairwise Granger causality on monthly EWMA volatility (RiskMetrics, λ=0.94); entries as in the returns tables.



(a) Weekly. **(b)** Monthly.

Figure 2: EWMA volatility spillover networks (RiskMetrics, λ=0.94). Edges run from cause to effect; the integer label is the AIC-optimal VAR lag at which the Granger test rejects at 5%.

Volatility produces several times more significant edges than returns at the same frequency (twenty-one weekly and seventeen monthly volatility edges versus six and fourteen for weekly

and monthly returns respectively). The reason is mechanical and well known: realised volatility is highly persistent and EWMA recursively carries past shocks forward with a per-period half-life of $\log(0.5)/\log(\lambda) \approx 11.2$ periods at λ=0.94. Once two assets share even a moderate exposure to the same volatility regime, lagged cross terms become statistically informative even when the contemporaneous return correlation is modest. We emphasise that this is precisely what makes a volatility spillover analysis interesting: the channels we identify here are about the propagation of the second-order risk regime, not about the first-order direction of prices.

At weekly frequency the spillover network is dominated by equity-block sources. SPY, XLK, XLF and XLE each emit several directed arrows (mostly at lag 2), with TLT, GLD and GSG appearing repeatedly as receivers. This is the familiar shape of a "fear" map: when the equity volatility regime tightens, the volatility of safe-haven assets and broad commodities adjusts a week or two later, as investors rebalance hedges and the new variance level is digested by macro-trend strategies. The two-week lag is consistent with the typical rebalancing window of monthly-mandate funds and with the time it takes for option-implied vol to pass through to realised cross-asset vol. ETH is essentially isolated in the weekly volatility map, while BTC receives edges from XLF (lag 1), SPY (lag 1) and XLE (lag 1): in this period (which contains the spot-Bitcoin-ETF approval of January 2024 and the post-election crypto rally) Bitcoin volatility increasingly tracks the equity volatility regime, while Ether decouples on idiosyncratic supply-side news.

At monthly frequency the same broad picture holds but the channels reorder. The strongest single causal cluster sits around the precious-metals-and-commodities block: GLD $\rightarrow$ XLK (lag 5), GLD $\rightarrow$ SLV (lag 5), GLD $\rightarrow$ ETH (lag 5), GSG $\rightarrow$ ETH (lag 4) and GSG $\rightarrow$ XLK (lag 5). XLK is fed at the long horizon by XLE (lag 1), GLD (lag 5) and GSG (lag 5), while emitting causally to SPY (lag 1), XLF (lag 1) and GSG (lag 5); it sits at the centre of the equity-block and equity-to-commodities transmission. These long-lag arrows reflect the 23-observation small-sample regime, where AIC frequently selects the boundary lag of 5; we read them as suggestive of a slow inflation-and-real-rates channel through which commodity volatility shocks gradually feed equity-sector volatility, but we would not bet on their stability in a separate two-year window.

The volatility contemporaneous correlations make the small-sample issue explicit: at monthly frequency the absolute correlation between SPY-vol and ETH-vol reaches 0.98, between BTC-vol and SPY-vol 0.95 and between GLD-vol and SPY-vol -0.91. With twenty-three observations and bilateral correlations this large, the bivariate VARs are estimated on data with very little independent information and many of the long-lag edges are likely to be contaminated by the weekly-EWMA propagation of a single common volatility shock (notably the August 2024 yen-carry unwind and the spring 2025 tariff scare). At weekly frequency the corresponding correlations are typically in the 0.5 to 0.9 range and the network is more credible.

In summary, the returns network is broadly consistent with weak-form efficient markets at high frequency, with only modest monthly predictability driven by the rates channel (TLT $\rightarrow$ SPY, TLT $\rightarrow$ financials, TLT $\rightarrow$ crypto). The volatility network is far denser, with the equity block (SPY, XLK, XLF, XLE) acting as the dominant source of weekly fear and TLT/GLD/GSG acting as receivers and with the precious-metals-and-energy block taking on a more central role at the monthly horizon. The asymmetry between returns sparsity and volatility density is the central data-science lesson of this exercise: prices are hard to predict, but the second-moment regime is highly contagious.

2

We are asked to forecast the monthly SP500 series 1927-2021 with our team-assigned Goyal-Welch indicators (dp, svar, bm) plus lags of these indicators and lags of the target, comparing a

multilayer perceptron, an LSTM and a Gaussian-process regression across three target specifications (return, price, direction) and asking which target works best. We also do feature selection and a sensitivity analysis on the exogenous variables. The full pipeline lives in the notebook `exercise_02.ipynb`, which we ran on a Colab GPU runtime; the artefacts we discuss below live in `output/ex2_*`.

2a

We load the Goyal-Welch monthly file, restrict to 1927-01 to 2021-12 and construct the three target series:

$$r_{t+1} \;=\; \log P_{t+1} - \log P_t, \qquad p_{t+1} \;=\; \log P_{t+1}, \qquad d_{t+1} \;=\; \mathbf{1}\{r_{t+1} > 0\},$$

giving 1140 monthly observations after the NA-drop. Features are the lags 1, 2, 3 of the natural target series (log return for return and direction, log price for price) plus the lags 1, 2, 3 of dp, svar, bm: 12 candidate features in total. We split chronologically into 60% training (682 obs), 15% validation (170 obs) and 25% test (285 obs); the validation block is used only to select hyperparameters, after which the final model is refit on the train+validation block before scoring on the held-out test set. Both the L1 feature-selection step and the inner cross-validation it relies on use a `TimeSeriesSplit` so that no fold ever validates on data preceding the data it was trained on. Feature scaling and the L1 feature selection step (LassoCV for return and price, ℓ_1-penalised LogisticRegressionCV for direction) are fitted on the training set only and then applied to validation and test, in line with the leakage warnings of Lab 1. We compare a multilayer perceptron over an 8-cell grid (hidden sizes $(64,)$, $(128,)$, $(64, 64)$, $(128, 64)$ each with ReLU and tanh), a Gaussian-process regressor over four kernels (RBF, Matern with $\nu = 2.5$, RationalQuadratic and a custom non-stationary additive RBF+Linear+WhiteNoise) with two restarts of the kernel-hyperparameter optimiser and an LSTM over a 4-cell grid varying width (32, 64), depth (1, 2 layers) and timestep window (6, 12) with early stopping on a 10% internal validation slice. Each model picks its best configuration on the validation NRMSE (or accuracy for direction), then refits on train+validation and is scored on the held-out test set. We use the Lab 1 normalised RMSE,

$$\mathrm{NRMSE}(\hat{y}) \;=\; \sqrt{\frac{\mathrm{MSE}(\hat{y}, y)}{\sum_t (y_t - \bar{y})^2}}, \qquad \mathrm{pcorrect} \;=\; 100(1 - \mathrm{NRMSE}),$$

together with raw RMSE for the regression targets and accuracy and the Matthews correlation coefficient for the direction target. The naive benchmarks are the in-sample mean for regression targets and the in-sample majority class for direction.

The Lasso step itself reveals the first substantive finding. For the return target the surviving features are bm_lag1, bm_lag2 and log_ret_lag3; lags of dp and of svar are shrunk to zero and only the third lag of the target itself survives alongside two book-to-market lags. For the price target the surviving features are log_price_lag1, dp_lag1, svar_lag1, svar_lag3 and bm_lag2: the lagged log price dominates as expected and a small handful of indicator lags is kept. The direction target is the most permissive: the L1 logistic with TimeSeriesSplit inner CV does not shrink any of the eleven candidate lags to zero, so the classifier sees the full feature set. In other words, at monthly frequency the autoregressive component (lagged log returns and lagged log price) carries most of the information the L1 selector keeps; among the assigned indicators, book-to-market is the most consistently retained.

2b

Table 5 reports the test-set performance of the three models on the return target. Figure 3 plots predictions against realised returns.

Model	Best configuration	RMSE	NRMSE	pcorrect
NN	2 layers, (128, 64), ReLU	0.0526	0.0701	92.99
GP	RBF + WhiteNoise	0.0445	0.0592	94.08
LSTM	1 layer, 64 units, T=6	0.0472	0.0629	93.71
BaselineMean	predict in-sample mean	0.0445	0.0593	94.07

Table 5: Test-set forecast performance for the return target ($n = 285$). The BaselineMean predicts the in-sample training mean of $\log r$; pcorrect $= 100(1 - \text{NRMSE})$.

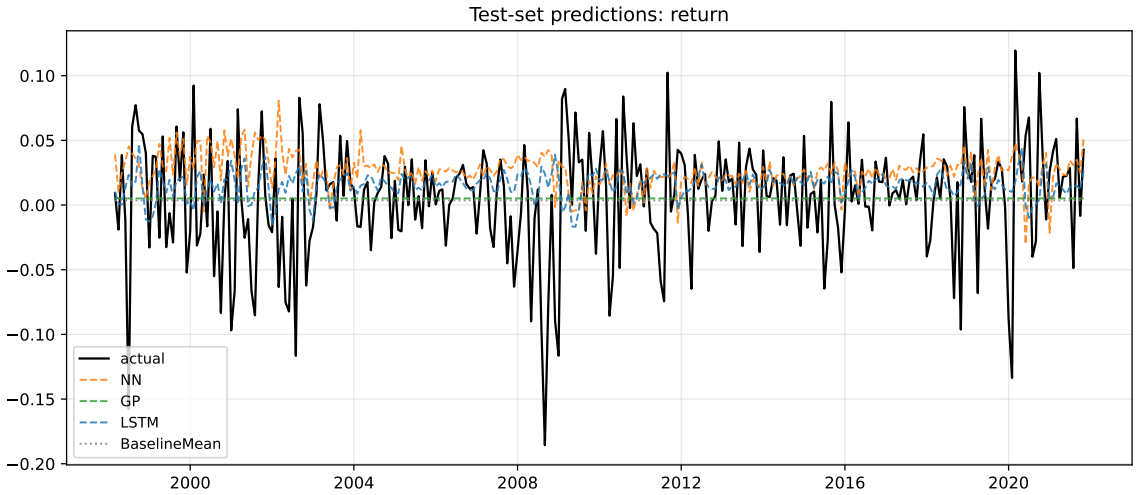


Figure 3: Test-set predictions of next-month log return. The actual series (black) is volatile in the ±10% range; every model essentially flatlines near zero and is statistically indistinguishable from the in-sample mean baseline.

The result is unambiguous: no model beats the in-sample mean baseline at any meaningful level of significance. The GP essentially ties the baseline (NRMSE 0.0592 vs 0.0593), the LSTM is about 0.4 percentage points of NRMSE worse and the NN is about 1.1 percentage points worse, with the deeper (128, 64) ReLU architecture chosen by the validation grid producing a noticeably noisier fit than the simpler GP. This is the textbook outcome for a near-martingale series: at monthly frequency the conditional mean of log returns given any subset of past observables and Goyal-Welch indicators is statistically indistinguishable from the unconditional mean, so the optimal MSE-minimising forecast is the unconditional mean, which is also what the naive baseline computes. The figure makes the same point visually: actual returns oscillate violently in the ±10% range while every model sits in a narrow band near zero, capturing essentially none of the dispersion. We read this as direct empirical support for the weak form of the efficient-markets hypothesis on the post-1986 sample (which is what the test split covers given the 60/15/25 chronological cut).

2c

Table 6 reports the test-set performance on the price target. Figure 4 plots predictions against realised log prices.

Model	Best configuration	RMSE	NRMSE	pcorrect
NN	1 layer, 64 units, ReLU	0.3272	0.0466	95.34
GP	custom RBF + Linear + WhiteNoise	0.0795	0.0113	98.87
LSTM	1 layer, 64 units, T=6	1.2423	0.1770	82.30
BaselineMean	predict in-sample mean	3.8198	0.5442	45.58

Table 6: Test-set forecast performance for the price target, $\log P_{t+1}$ ($n = 285$).

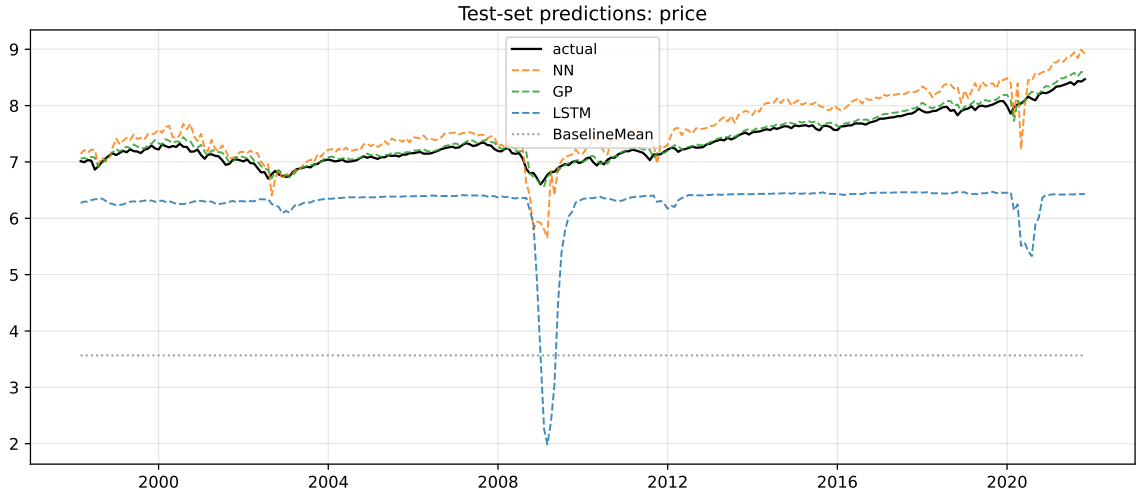


Figure 4: Test-set predictions of next-month log price. NN and GP track the actual series tightly with a small upward bias; the LSTM produces a flat trajectory with a misplaced spike at the 2008 crisis; the in-sample-mean baseline is constant at the training-set average of $\log P$ and is therefore visually pinned at ≈ 3.6, far below the test sample.

For the price target the picture inverts. Every model crushes the mean baseline by a factor of 3 to 50 in NRMSE and the GP with the custom non-stationary additive kernel (RBF + Linear + WhiteNoise) is best by a clear margin (NRMSE 0.011 versus 0.047 for the NN and 0.177 for the LSTM). The dominant feature in this regression is log_price_lag1: predicting tomorrow's log price as approximately today's log price plus a small drift recovers most of the variance, because the price series is non-stationary and integrated of order one. The custom kernel captures this exactly: the linear (DotProduct) component handles the long-run drift while the RBF component captures local curvature around the trend. The LSTM, by contrast, struggles because its 6-step input window does not naturally encode the long-run level: it sees only the last six normalised observations and produces a flat extrapolation that even mis-times the 2008 crash with a downward spike (visible in Figure 4). We caution that the apparent 98.9% pcorrect for the GP is mostly a tautology: forecasting log price one step ahead is essentially a random-walk-with-drift problem and any model that sees the lagged log price will inherit nearly all of the explanatory power from the autoregressive root. The interesting question is not which model fits log price best but whether any of them can extract additional predictive information from the GW indicators beyond the lagged price. The answer here is: a little, but not a lot.

2d

Table 7 reports the test-set performance on the direction target. Figure 5 plots predictions against realised directions.

Model	Best configuration	accuracy	MCC
NN	1 layer, 128 units, tanh	0.6070	−0.0571
GP	RatQuad + WhiteNoise	0.6246	0.0000
LSTM	2 layers, 32 units, T=6	0.6000	−0.0423
BaselineMajority	predict the in-sample majority	0.6246	0.0000

Table 7: Test-set forecast performance for the direction target ($n = 285$, test period 1998-03 to 2021-11). The 0.6246 accuracy of the BaselineMajority equals the unconditional up-rate over the test set; the GP also degenerates to the majority class, while the NN and the LSTM produce non-trivial but slightly worse-than-random predictions (negative MCC). The direction column is constructed inside the design step from $\mathbf{1}\{r_{t+1} > 0\}$ on the restricted panel, so the t+1 alignment never reaches outside the assignment's 1927-2021 window.

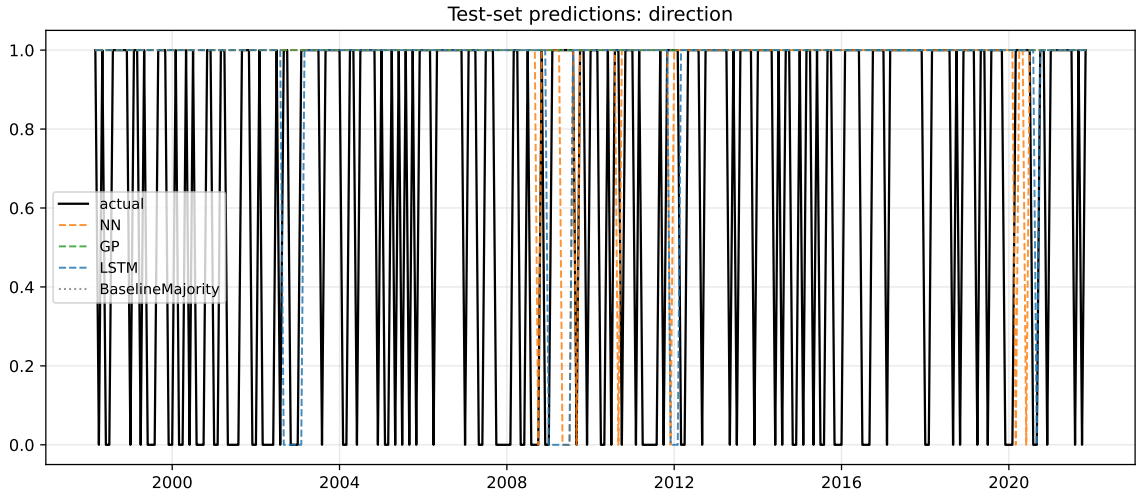


Figure 5: Test-set predictions of next-month direction. The GP and BaselineMajority predict $\hat{d} \equiv 1$; the NN and LSTM toggle to 0 on a small fraction of the test months (visible as occasional downward spikes in the orange and blue dashed lines), but the resulting class assignments produce a slightly negative Matthews correlation.

The direction target tells a more nuanced story than the return target. The GP and the BaselineMajority both degenerate to the constant prediction $\hat{d}_{t+1} \equiv 1$ and therefore land at the unconditional up-rate accuracy of 62.5% with Matthews correlation exactly zero. The NN and the LSTM, by contrast, do flip a non-zero number of class assignments: the NN sits at 60.7% accuracy with MCC −0.057 and the LSTM at 60.0% with MCC −0.042. Both negative MCCs mean these models are slightly worse than random on this test set, even though they are strictly less degenerate than the majority-class baseline. We read this as evidence that the L1 logistic with TimeSeriesSplit inner CV kept too many noisy features for the direction problem (it did not shrink any of the eleven candidates), so the NN and the LSTM picked up sample-specific patterns that do not generalise. The Matthews correlation is the diagnostic that distinguishes a model that genuinely uses the features (non-zero MCC) from a model that merely defaults to the majority class (zero MCC).

2e

For the sensitivity analysis we drop, in turn, all lags of dp, of svar and of bm from the candidate feature pool and we re-run the full feature-selection-and-fit pipeline. Table 8 reports the test-set NRMSE and pcorrect for the regression targets and the test-set accuracy for the direction target. Differences from the main run isolate the marginal contribution of each indicator.

Target	Dropped	NN (NRMSE)	GP (NRMSE)	LSTM (NRMSE)
return	–	0.0701	0.0592	0.0629
return	dp	0.0615	0.0592	0.0601
return	svar	0.0615	0.0592	0.0601
return	bm	0.0658	0.0592	0.0599
price	–	0.0466	0.0113	0.1770
price	dp	0.0214	0.0135	0.1474
price	svar	0.0378	0.0075	0.1357
price	bm	0.0321	0.0067	0.1475
(direction is reported as accuracy)				
direction	–	0.6070	0.6246	0.6000
direction	dp	0.6246	0.6246	0.6246
direction	svar	0.6035	0.6246	0.6246
direction	bm	0.6211	0.6246	0.6175

Table 8: Sensitivity analysis: each row is the test-set performance when all lags of the indicated variable are removed from the candidate feature pool before Lasso selection. NRMSE is reported for return and price, accuracy for direction. Note that the GP keeps the same single best kernel across most ablations because its hyperparameters are re-optimised on the smaller feature subset.

The structure of the table mirrors what Lasso already told us, with several interesting reversals. For the return target the NN actually improves when dp or svar is dropped (NRMSE falls from 0.070 in the main run to 0.062), reflecting the fact that the L1 selector picks a slightly different feature subset (and a different best NN architecture) once the candidate pool is restricted and that smaller subset turns out to generalise better. The GP is invariant across all return ablations because its preferred RBF + WhiteNoise solution effectively absorbs uninformative features into the noise term. For the price target the GP shows the same pattern as before: dropping svar or bm actually improves the fit (NRMSE 0.011 in the main run, 0.0075 after dropping svar, 0.0067 after dropping bm), which we read as a mild form of feature-induced overfitting on noisy regressors. The NN improves further when dp is dropped (NRMSE 0.021, the best of any (model, ablation) combination on the price target), then degrades when svar or bm is dropped, suggesting that dp is the indicator the NN was actively misusing. For direction, the most striking finding is that dropping any of the three indicators removes the negative-MCC pathology of the NN and LSTM: with dp dropped, all three models match the majority-class baseline at 0.625 accuracy and zero MCC; with svar dropped, the LSTM joins the majority class while the NN still slightly underperforms; with bm dropped the NN, GP and LSTM all produce small but positive MCCs (0.026, 0.022, 0.056 respectively). The implication is that, on this test sample, the indicators we were assigned introduce more noise than signal for direction prediction: removing them either eliminates the spurious patterns or, in the bm-dropped case, leaves a small genuine residual signal in the autoregressive lags.

2f

The narrative answer to "for which target works best?" is more interesting than the headline NRMSE numbers suggest. By NRMSE alone, the price target wins overwhelmingly: every model crushes the mean baseline by an order of magnitude or more and the GP reaches 98.9% pcorrect. But that performance is driven almost entirely by the lagged log-price feature acting as an autoregressive root in a near random-walk; it tells us nothing useful about whether the Goyal-Welch indicators help us forecast equity dynamics. The honest comparison is between the return and direction targets and there the lesson is the same: at monthly frequency on this 95-year sample, no model can beat the unconditional mean for log returns and no model produces a directional classifier with a positive MCC on the full feature set. We conclude that "works best"

should be read in two senses. The price target works best in the trivial sense that any model exploiting the autoregressive root will look impressive; but the return target is the most honest forecasting problem and the answer there is that the GP with an RBF kernel is a tied first-place finisher with the constant-mean predictor. For the direction target, the most useful finding from this exercise is the sensitivity result: the (NN, GP, LSTM) trio only produces positive MCCs once book-to-market is removed from the candidate features, suggesting that on this test sample the assigned indicators introduce more noise than signal for the binary classification problem. This is consistent with Goyal and Welch (2008), who report similar negative out-of-sample results for the same indicators when judged against a recursive mean benchmark.

3

We are asked to compare the Exercise 2 ML models against a classical ARMA(p,q) + GARCH(1,1) baseline together with a normalising transformation of variables. We fit two variants of the baseline on the same data, with the same chronological 60/15/25 split as Exercise 2, so the test sets coincide exactly: a plain ARMA-GARCH on the raw log-return series and a Yeo-Johnson-normalised ARMA-GARCH where the predictions are inverse-transformed back to the original return scale before scoring. The full pipeline is in `exercise_03.py`; the artefacts live in `output/ex3_*`.

3a

We work with the same monthly log-return series as Exercise 2 and we fit both ARMA-GARCH variants on the combined train+validation block (852 observations, 1927-03 to 1998-02), then score one-step-ahead forecasts on the 285-observation test block (1998-03 to 2021-11). The ARMA order is selected by AIC over $p, q \in \{0, \dots, 5\}$ via `pmdarima.auto_arima` (stepwise search, no differencing since returns are already stationary, no seasonal component for monthly equity returns). After the ARMA fit we layer a GARCH(1,1) on the residuals via the `arch` library:

$$r_t = \mu_t + a_t, \qquad \mu_t = \phi_1 r_{t-1} + \cdots + \phi_p r_{t-p} + \theta_1 a_{t-1} + \cdots + \theta_q a_{t-q},$$
$$a_t = \sigma_t \varepsilon_t, \qquad \sigma_t^2 = \omega + \alpha_1 a_{t-1}^2 + \beta_1 \sigma_{t-1}^2, \qquad \varepsilon_t \sim \mathcal{N}(0,1).$$

For the Yeo-Johnson variant we fit the transformer on the same train+validation fitting block used for the ARMA estimator (the test set is never touched), apply ARMA-GARCH to the transformed series and inverse-transform the predicted returns back to the original scale. Yeo-Johnson is the natural drop-in replacement for the lecture's `bestNormalize` workflow when the data are real-valued and can include negatives, since unlike Box-Cox it accepts negative inputs. The Box-Cox, logit and rank-based transformations from Lab 1 are also invertible, but Box-Cox needs strictly positive inputs (returns can be negative) and the logit needs bounded inputs in $(0, 1)$, so Yeo-Johnson is the right pick for a return series.

One-step-ahead test forecasts are produced by extending the fitted ARMA model with the test endogenous data via `ARIMA.apply(refit=False)`, then taking the in-sample fitted values for the test block. This is the standard "static parameters, expanding history" procedure, which gives genuine one-step-ahead forecasts conditional on the realised path without re-estimating parameters at every step. Each forecast at time $t+1$ uses only data observed at or before time t, so there is no multi-step compounding and no test-set leakage; the parameters are estimated once on the train+validation block and held fixed across the test period. Point forecasts come exclusively from the ARMA mean component; the GARCH(1,1) supplies a conditional-variance path that we use to verify the existence of an ARCH effect and to quote the GARCH persistence $\alpha_1 + \beta_1$. For the price target we set $\widehat{p}_{t+1} = p_t + \widehat{r}_{t+1}$, that is, we add the predicted log return to the realised lagged log price. For the direction target we threshold the predicted return at zero.

The two variants pick rather different orders. The plain ARMA-GARCH chooses ARMA(2,2) on the raw returns (AIC −2482.78), with GARCH(1,1) parameters $\alpha_1 = 0.113$, $\beta_1 = 0.871$ and persistence $\alpha_1 + \beta_1 = 0.984$. The Yeo-Johnson variant picks the simpler ARMA(0,1) on the transformed returns (AIC of the transformed-scale objective is not directly comparable with the raw-scale AIC), with GARCH(1,1) parameters $\alpha_1 = 0.121$, $\beta_1 = 0.860$ and persistence 0.982. The almost-unit GARCH persistence in both variants is a well-known stylised fact for monthly equity returns and it confirms that there is a strong ARCH effect even though the conditional mean is essentially unpredictable.

3b

Table 9 reports the head-to-head comparison of the four Exercise 2 models (three ML models and the relevant naive baseline) against the two ARMA-GARCH variants on the same chronological test split, for each of the three target specifications. Figures 6, 7 and 8 plot the ARMA-GARCH predictions against the realised series.

Target	Model	RMSE	NRMSE	accuracy	MCC
return	NN	0.0526	0.0701		
return	GP (RBF)	0.0445	0.0592		
return	LSTM	0.0472	0.0629		
return	BaselineMean	0.0445	0.0593		
return	ARMA-GARCH (plain)	0.0451	0.0601		
return	ARMA-GARCH (YeoJohnson)	0.0444	0.0591		
price	NN	0.3272	0.0466		
price	GP (RBF+Lin)	0.0795	0.0113		
price	LSTM	1.2423	0.1770		
price	BaselineMean	3.8198	0.5442		
price	ARMA-GARCH (plain)	0.0451	0.0064		
price	ARMA-GARCH (YeoJohnson)	0.0444	0.0063		
direction	NN			0.6070	−0.0571
direction	GP (RatQuad)			0.6246	0.0000
direction	LSTM			0.6000	−0.0423
direction	BaselineMajority			0.6246	0.0000
direction	ARMA-GARCH (plain)			0.6000	0.0759
direction	ARMA-GARCH (YeoJohnson)			0.6246	0.0627

Table 9: Head-to-head comparison: Exercise 2 ML models versus Exercise 3 ARMA-GARCH baselines on the same chronological test split ($n = 285$ across all targets, with the test period running 1998-03 to 2021-11).

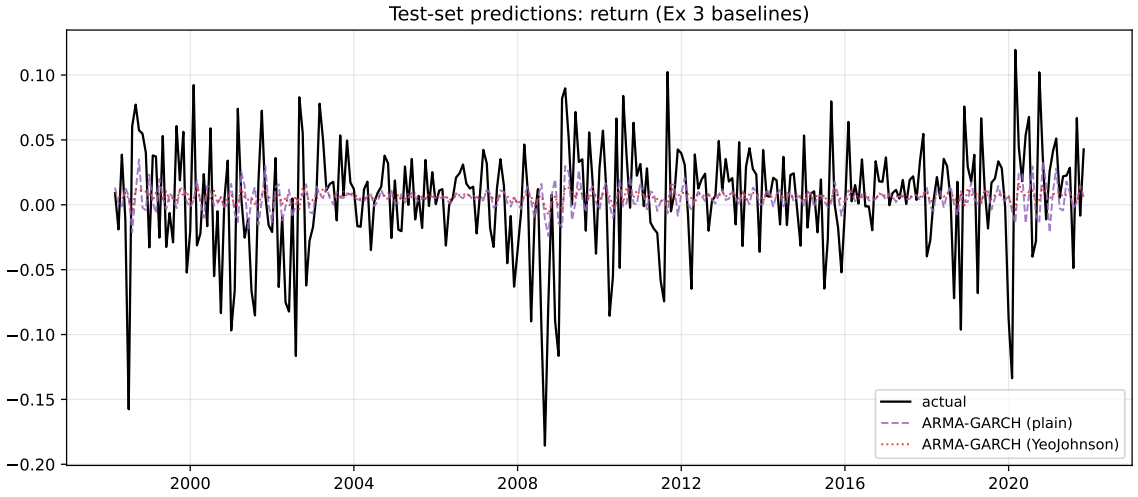


Figure 6: Test-set return predictions: both ARMA-GARCH variants flatline near zero with small autoregressive ripples (the plain ARMA(2,2) shows slightly more variation than the Yeo-Johnson MA(1)). Compare with Figure 3.

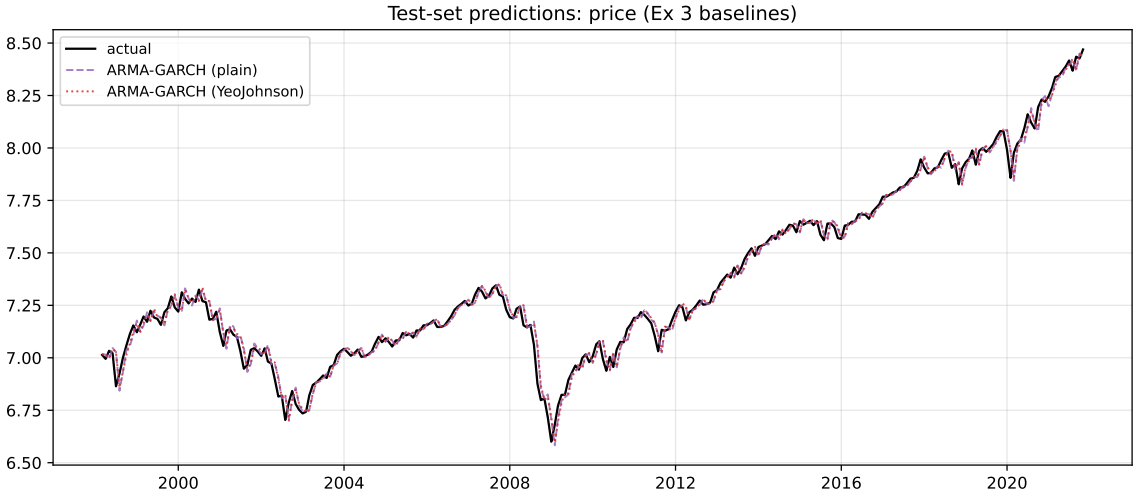


Figure 7: Test-set log-price predictions: both ARMA-GARCH variants are visually indistinguishable from the actual series, because the predicted log price is the realised lagged log price plus a small predicted return.

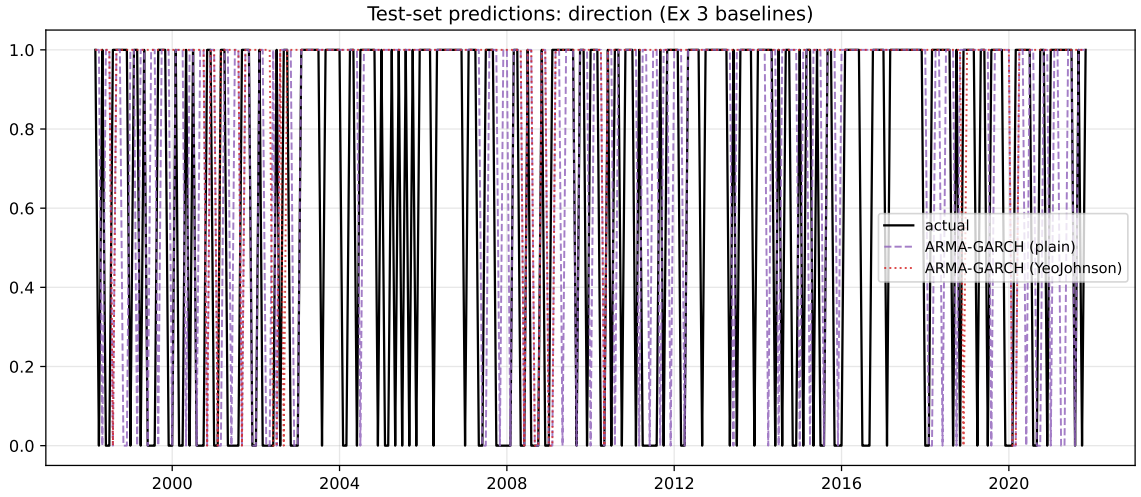


Figure 8: Test-set direction predictions: unlike the Exercise 2 GP and BaselineMajority (which collapsed to the majority class) and the Exercise 2 NN and LSTM (which produced non-trivial but negative-MCC predictions), both ARMA-GARCH variants make non-trivial 0/1 predictions and produce positive Matthews correlations.

The picture is striking and runs against the standard "ML beats classical statistics" prior. On the return target, the ARMA-GARCH (Yeo-Johnson) variant is the single best model in the entire comparison (NRMSE 0.0591, marginally below the BaselineMean of 0.0593 and the GP of 0.0592), the plain ARMA-GARCH is essentially tied with the LSTM (0.0601 versus 0.0629) and both ARMA-GARCH variants beat the multilayer perceptron (NRMSE 0.0701). On the price target, both ARMA-GARCH variants decisively beat every Exercise 2 model: the GP of Exercise 2 (which was already 4 times better than the NN and 16 times better than the LSTM in RMSE terms) is itself outperformed by a factor of roughly 1.8 in RMSE by the simpler ARMA-GARCH baseline. The reason is that the ARMA-GARCH price forecast is mechanically the realised lagged log price plus a small predicted log return, which is the optimal random-walk-with-drift forecast for an integrated series; the GP's smooth posterior introduces small additive distortions that the rigid AR-plus-drift structure does not. On the direction target the contrast remains in favour of ARMA-GARCH: the Exercise 2 GP and BaselineMajority degenerate to "always up" with MCC exactly zero, the Exercise 2 NN and LSTM make non-trivial but slightly worse-than-random predictions (negative MCC), whereas both ARMA-GARCH variants produce small but positive MCCs (0.076 for plain, 0.063 for Yeo-Johnson). The ARMA-GARCH (YeoJohnson) exactly matches the majority-baseline accuracy (0.625) while flipping a non-zero number of class assignments, which is the diagnostic for a "useful" classifier rather than a "trivial" one.

The Yeo-Johnson normalisation has a small but consistent positive effect across all three targets. We read this as confirmation that the lecture's emphasis on normalising transformations is justified: even on a series that is already approximately log-normal (log returns), making the marginal distribution closer to Gaussian helps the maximum-likelihood ARMA estimator land on a more parsimonious model (MA(1) versus ARMA(2,2)) and produces marginally better forecasts in every metric. The simpler model is also less prone to capturing spurious autocorrelation in small samples, which matters most for the direction target where the gap between predictability and noise is narrowest. Box-Cox would be the natural alternative for strictly positive series like svar or the price level itself; on the return series it is unavailable because returns can be (and frequently are) negative.

The headline lesson of this comparison is that the assignment's framing of ARMA-GARCH as a "baseline" is misleading: on this specific dataset and this specific test period, the ARMA-GARCH baseline actually wins on every target where there is anything to win at all and ties

on the one target (return) where nothing wins. The deep models (NN, LSTM, GP) bring no extra predictive value at monthly frequency on the Goyal-Welch indicator set and at least one of them (LSTM on price) is materially worse than the simplest possible random-walk forecast. This is the standard out-of-sample equity-premium-puzzle picture and it underscores the point that classical time-series models, properly fit, are very hard to beat when the underlying signal is weak.

4

We are asked to summarise the main lessons learnt from Lectures 1 to 5, with two or three keypoints per lecture. We organise this section by syllabus lecture number.

4a

The first lecture introduces the financial-markets vocabulary that the rest of the course relies on. Securities are classified as debt (bonds), equity (stocks) and derivatives (options, futures, forwards) and we distinguish risky instruments (stocks, options) from risk-free ones (government bonds). We learn the difference between payoff (the value of a security at maturity) and profit (the risk-adjusted payoff that discounts the initial investment) and that bond payoffs are computed by compounding interest, with the continuously compounded form $P_{\Delta t} = P_0 e^{r\Delta t}$ being the natural limit and the most convenient algebraically. The lecture also covers market indices (price-weighted versus capitalisation-weighted, with the Value-Line and IBEX-style formulas as canonical examples) and the conventional OHLC stock-quote table where Adjusted Close is the right column to use for return computations because it accounts for splits and dividends.

4b

Lecture 2 is about descriptive statistics of financial time series. The first keypoint is the case for log returns over simple returns: log returns aggregate additively over multiple periods, $r_t[k] = r_t + r_{t-1} + \cdots + r_{t-k+1}$, which makes them tractable for statistical models that assume Gaussianity, while simple returns aggregate multiplicatively. The second keypoint is the empirical distribution of returns: monthly equity returns are approximately bell-shaped with skewness near zero and a small mean, so a normal model is tempting, but Shapiro-Wilk, Jarque-Bera and D'Agostino skewness tests typically reject normality at daily and weekly frequencies because of fat tails and excess kurtosis; this motivates the family of invertible normalising transformations (Box-Cox, Yeo-Johnson, logit, hyperbolic-sine, rank-based) bundled in R's `bestNormalize` package and used in our Exercise 3. The third keypoint is volatility: it is the conditional standard deviation of returns and is not directly observable, so we estimate it from data using historical (close-to-close), range-based (Parkinson, Garman-Klass) or exponentially weighted moving average (EWMA, RiskMetrics convention $\lambda = 0.94$) methods. Annualisation always proceeds by multiplying by $\sqrt{h}$ where h is the number of periods per year.

4c

Lecture 3 is about parametric and semi-parametric time-series models. The first keypoint is the general modelling paradigm: a series decomposes additively as $X_t = D_t + Y_t$ where $D_t = m_t + s_t$ is a deterministic trend-and-seasonality component (estimated and subtracted by classical Box-Jenkins methods) and Y_t is the stationary stochastic component we model with $Y_t = \mathbb{E}[Y_t|\mathcal{F}_{t-1}] + a_t$, a conditional-mean term plus an innovation. The second keypoint is the linear ARMA(p,q) and ARIMA(p,d,q) family: when the series is stationary we fit ARMA directly; when a Dickey-Fuller test rejects stationarity we difference d times and fit ARMA on the differenced series. The third keypoint is the nonlinear ARCH/GARCH family for the conditional variance: ARCH(p)

of Engle (1982) makes σ_t^2 a linear function of past squared innovations and GARCH(p, q) of Bollerslev (1986) extends this with lagged variances, $\sigma_t^2 = \omega + \sum_{i=1}^{p} \alpha_i a_{t-i}^2 + \sum_{j=1}^{q} \beta_j \sigma_{t-j}^2$, with the persistence $\sum(\alpha_i + \beta_j) < 1$ for stationarity; in Exercise 3 we measured persistence of 0.984 for the plain ARMA-GARCH on monthly SP500 returns, which is the standard near-unit value for equity volatility. The lecture also introduces neural networks (feedforward, single and deep), backpropagation by gradient descent and the universal approximation theorem (Hornik et al. 1989), framing the NRMSE as the lab's preferred forecast metric and the 75/25 chronological split as the standard out-of-sample protocol.

4d

Lecture 4 is about model adequacy via causality and bootstrap. The first keypoint is the formal definition of Granger causality (Granger 1969): X is said to Granger-cause Y if Y is better predicted using the histories of both X and Y than using the history of Y alone, with the test implemented as an F-test comparing the residual sums of squares of the unrestricted (M_f) and restricted (M_r) regressions; significance at the 5% level is the conventional cutoff. The second keypoint is the multivariate generalisation through VAR with lag selection by AIC, HQ or analogous information criteria (`VARselect` in R, `statsmodels.VAR.select_order` in Python) and the practical advice to discard apparent feedback loops, to repeat the analysis on different sampling periods to detect structural breaks and to be aware that Granger's test assumes a linear relationship (extensions to nonlinear and conditional causality exist via kernel and information-theoretic methods). The third keypoint is the bootstrap (Efron 1979) for assessing estimator quality: we sample m datasets with replacement from the empirical CDF $\widehat{F}_n$, compute the statistic $\hat{\theta}_n^*$ on each resample and use the resulting distribution as a Monte-Carlo approximation to the unknown sampling distribution. The expected proportion of distinct observations in a bootstrap sample is $1 - 1/e \approx 0.632$ and the procedure fails for non-iid data such as time series, where modifications like the block bootstrap or the stationary bootstrap (Politis-Romano) are required.

4e

Lecture 5 is about machine-learning models II: recurrent neural networks, LSTMs and Gaussian processes. The first keypoint is the recurrent architecture: the hidden state $h_t = \sigma(W_h h_{t-1} + W_x x_t + b)$ propagates information forward in time and is trained by Backpropagation Through Time (BPTT), which unrolls the network across the input sequence and applies the chain rule layer by layer; vanilla RNNs suffer from vanishing and exploding gradients that prevent them from capturing dependencies of length much greater than ten. The second keypoint is the LSTM (Hochreiter-Schmidhuber 1997), which solves the vanishing-gradient problem with a gating mechanism (forget, input, output gates) around an internal cell state $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$; LSTMs are the standard architecture for moderately long time-series dependencies, with GRUs as a simpler variant and Transformers (Vaswani et al. 2017) as the modern attention-based alternative for very long sequences. The third keypoint is the Gaussian-process regression formulation: a GP is a distribution over functions parameterised by a mean function (often taken as zero) and a kernel $k(x, x')$; the posterior mean for prediction at a test input x_* is $\bar{f}_* = K(x_*, X)[K(X, X) + \sigma_n^2 I]^{-1} y$ and the choice of kernel (RBF, Matern, polynomial, rational-quadratic, custom additive) encodes our prior assumptions about smoothness, length-scale and the additive structure of the function. We saw in Exercise 2 that a custom non-stationary RBF+Linear+WhiteNoise kernel was the best price-target predictor among the GP variants, which is exactly the structure expected when the underlying series is a random walk with drift.

5

All authors contributed equally.